

Experiment 6

Arman Imam

EEE3342C

25Fall 0011

Equipment:

Computer with Xilinx ISE software, Vivado 2017.4 specifically, and Artix-7 FPGA Board

Objective:

The objective of experiment 6 is to introduce the user how latches and flip flops are utilized in Verilog. For part 1, the user is asked to create a gated and non gated SR latch, and test it in a test bench and FPGA board to be recorded. For part 2, the user is asked for the same information, except for the D flip-flop and D latch. Part 3 is the same story, but with a JK Flip flop; and part 4 with T Flip-Flops. Part 5 asks the user to create an 8 bit dual edge-triggered flip-flop. The edge is triggered by both posedge and negedge.

Design Steps:

I began by creating a single project file, where I created the specified source files (nongated SR latch, gated SR latch, D latch, D flip flop, JK flip flop, T flip flop, and 8 bit dual edge flip flop). I also created a test bench for each of source file, along with the constraints for the FPGA board.

Part 1:

Verilog RTL of gated & non-gated SR latch:

Gated SR latch: The source code for the gated SR latch had 3 inputs(S, R, enable), and 2 outputs (Q, Q_Bar). S and R are input signals, enable is a gated signal, and Q and Q_bar are output signals, where Q_bar is the complement of Q. In the code, Q+ is 0 when reset is 0 or Q is 0, and Q+ is 1 when set or Q is 1. If S and R are both 1, that results in an undefined. When enable is 0, Q will remain unchanged, and when enable is 1, it'll act as a normal SR latch.

```
1  `timescale 1ns / 1ps
2  //set reset latch, 3 inputs, 2 outputs
3  module gated_SR_latch(
4      input S, R, enable,
5      output reg Q, Q_bar
6  );
7      always @(*) begin
8          if(enable) begin
9              case ({S,R})
10                 //2'b00: Q <= Q; <-- Redundant
11                 2'b01: Q <= 0;
12                 2'b10: Q <= 1;
13                 2'b11: Q <= 1'bX;
14             endcase
15         end
16         Q_bar <= ~Q;
17     end
18 endmodule
```

Non-Gated SR latch: the source code for the nongated SR latch has 2 inputs(S, R), and 2 outputs(Q, Q_bar). It's essentially the same as the gated SR latch, except it doesn't have an enable gate signal.

```
1  `timescale 1ns / 1ps
2
3  module non_gated_SR_latch(
4      input S, R,
5      output reg Q, Q_bar
6  );
7      always @(*) begin
8          case({S, R})
9              //2'b00: Q <= Q; <--- Redundant,
10             2'b01: Q <= 0;
11             2'b10: Q <= 1;
12             2'b11: Q <= 1'bX;
13         endcase
14         Q_bar <= ~Q;
15     end
16
17 endmodule
```

Pictures of FPGA board: The board provided expected results. When enable was set to 0, Q remained the same, and enable set to 1 acted like a normal SR latch.

Gated SR latch:

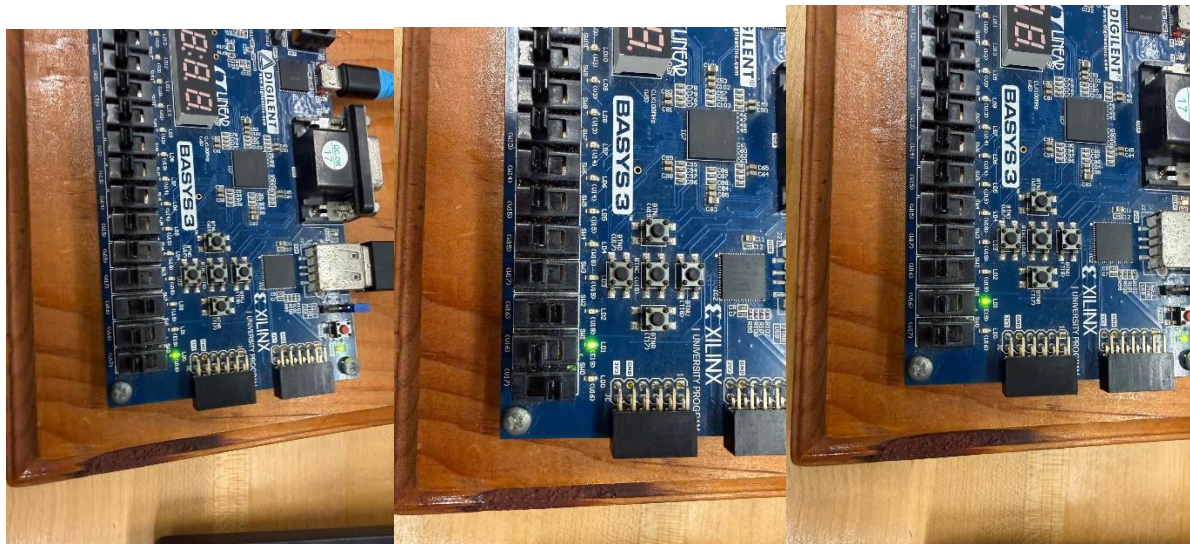
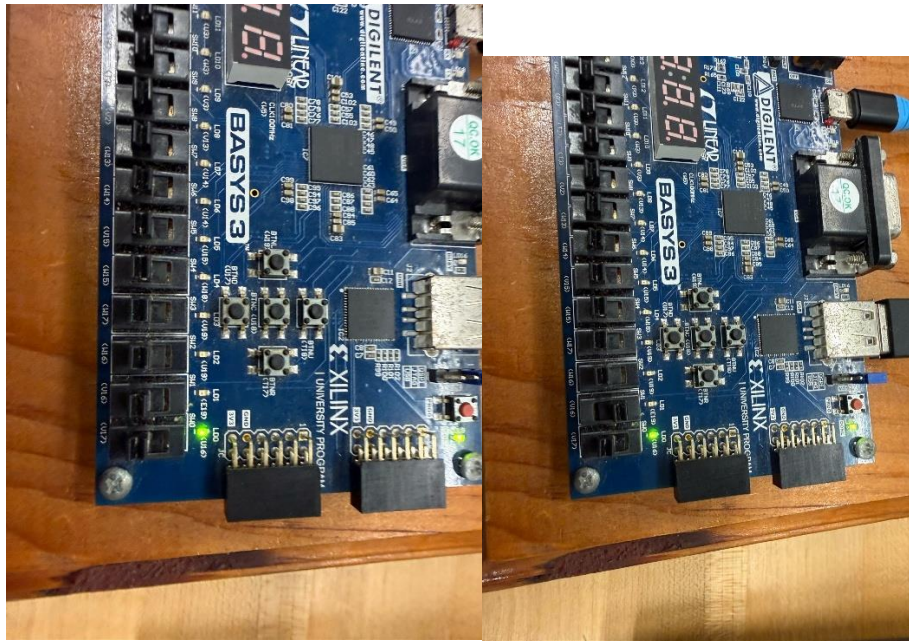
V17 = enable

V16 = S

W16 = R

U16 = Q

E19 = ~Q



Non-Gated SR latch: The board provided expected results. When S was enabled by itself, Q became 1, and When R was enabled by itself, Q became 0.

V17 = S

V16 = R

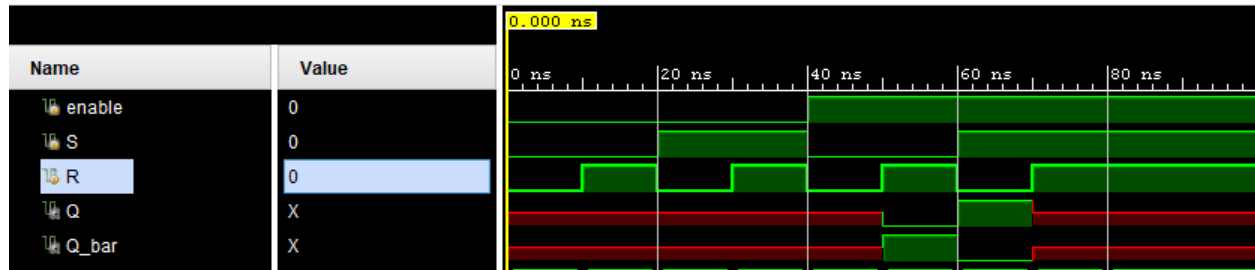
U16 = Q

E19 = $\sim Q$

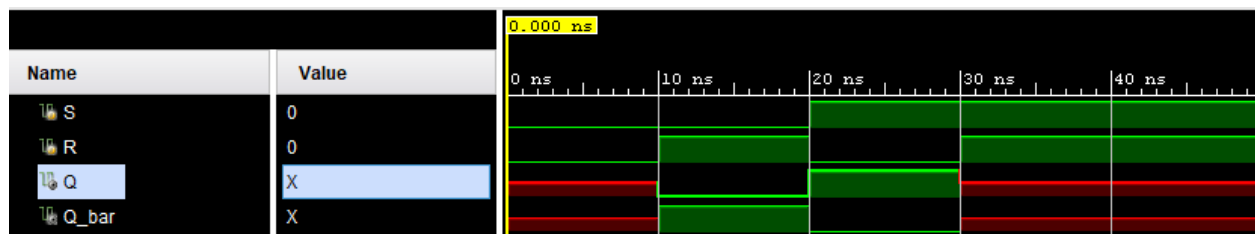


Screenshot of waveform that matches the above truth tables of SR-latches

Gated SR latch: When enable is set to 0, there is no change to Q; however Q hasn't been set to anything, so it shows undefined. When enable becomes 1, Q is set to 1 for set and 0 for 0. When S and R are set to 1, Q is shown to be undefined again. Values of Q_bar are the complement of Q, (unless undefined, then it's undefined)



Non-Gated SR latch: This waveform is the same as the gated SR latch, but when Enable is set to 1 indefinitely.



Part 2:

Verilog RTL of D flip-flop and D latch

D Flip-Flop: The source code for the D flip-flop has two inputs(D, clk), and 2 outputs(Q, Q_bar). D is a input signal, clk is a clock enable signal, and Q/Q_bar are output signals. Since the D flip flop is similar to the gated SR latch, I was able to call the gated_SR_latch module to reflect the D flip flop behavior.

```
1  `timescale 1ns / 1ps
2
3  module dflip_flop(
4      input D, clk,
5      output Q, Q_bar
6  );
7      gated_SR_latch ACT(.S(D), .R(~D), .enable(clk), .Q(Q), .Q_bar(Q_bar));
8
9  endmodule
```

D Latch: The source code for the D latch has two inputs(D, En), and 2 outputs(Q, Q_bar). The code for the D latch and D flip flop are essentially identical, except the D latch uses a gated signal(En), while the D flip flop uses a clock signal(clk).

```
1  `timescale 1ns / 1ps
2
3  module dlatch(
4      input D, En,
5      output Q, Q_bar
6  );
7      gated_SR_latch ADT(.S(D), .R(~D), .enable(En), .Q(Q), .Q_bar(Q_bar));
8  endmodule
```

Pictures of FPGA board for D flip-flop **CLOCK PIN is W5**

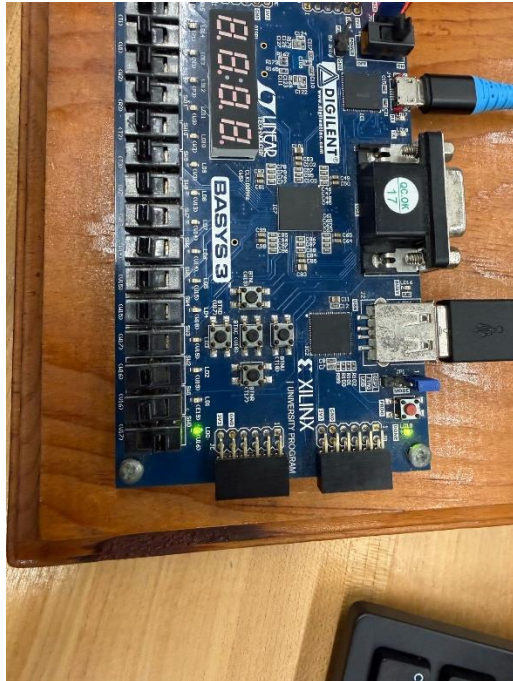
The FPGA board gives the expected results. When D = 1, Q = 1, and D = 0, Q = 0.

W5 = Clk

V17 = D

U16 = Q

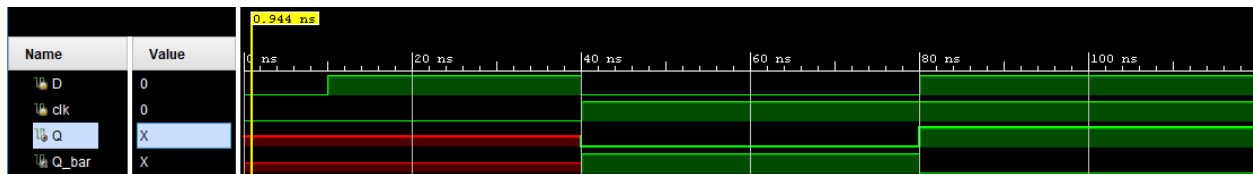
E19 = ~Q



2-3 Sentence of test plan

The test plan for the dfflipflop begins by setting clk and D = 0, and after 10 nanoseconds, I've set D = 1. After 30 nanoseconds, I've set clk = 1 and D = 0. After 40 seconds I've set D = 1, with another 40 seconds before ending the waveform. The results of Q and Q_bar are as expected.

Waveform based on test plan



Part 3:

- Verilog RTL of JK flip-flop: The source code for the JK flip flop uses 3 inputs(J, K, clk) and 2 outputs(Q, Q_Bar). The JK flip flop acts as a SR flip flop, but the only difference is that when S and R are equal to 1, Q is toggled. In this case, I've set Q_Bar in a combinational logic style, so that after Q is set, Q_Bar is set afterwards.

```

1  `timescale 1ns / 1ps
2
3  module jk_flip_flop(
4      input J, K, clk,
5      output reg Q,
6      output Q_Bar
7  );
8      always @(posedge clk) begin
9          case ({J, K})
10             2'b00: Q <= Q;
11             2'b01: Q <= 0;
12             2'b10: Q <= 1;
13             2'b11: Q <= ~Q;
14         endcase
15     end
16     assign Q_Bar = ~Q;
17 endmodule

```

- Pictures of FPGA board for JK flip-flop: The FPGA board gives the expected results. When J and K are equal to 1, Q toggles to the next iteration.

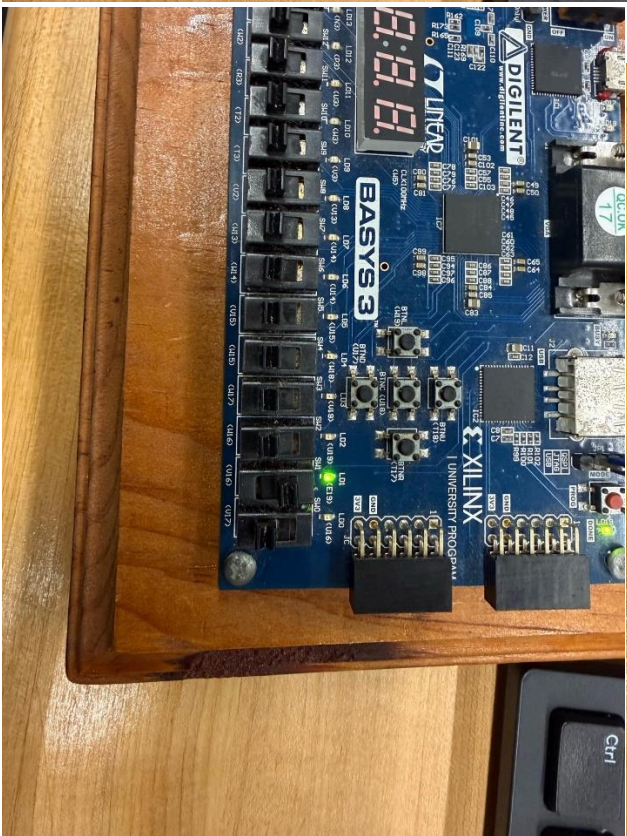
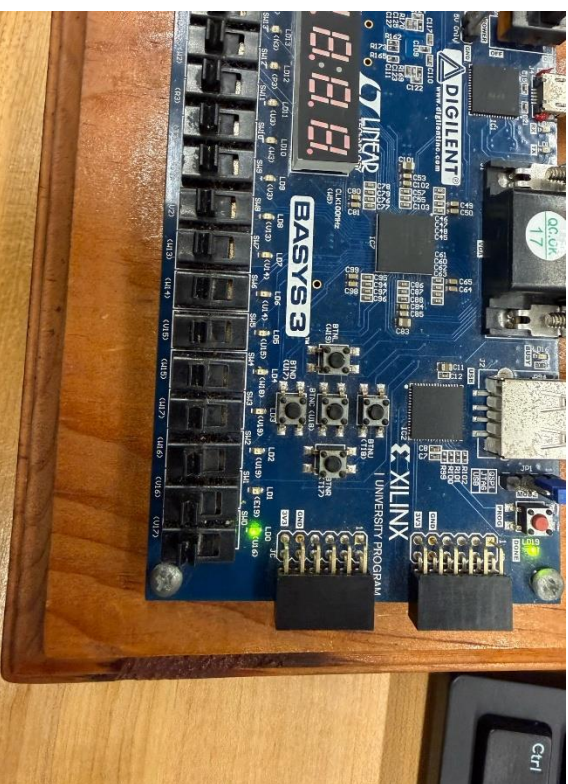
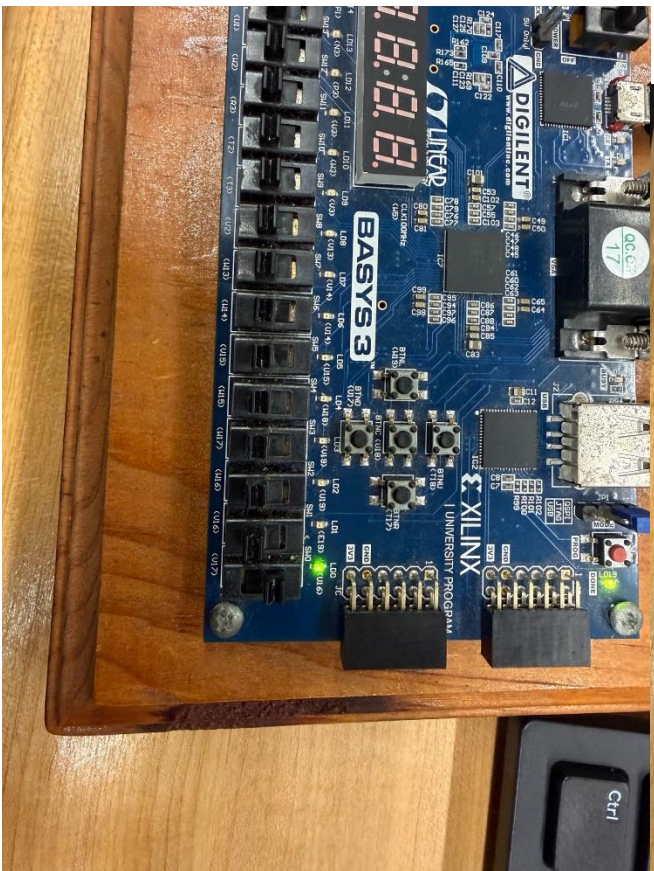
W5 = Clk

V17 = J

V16 = K

U16 = Q

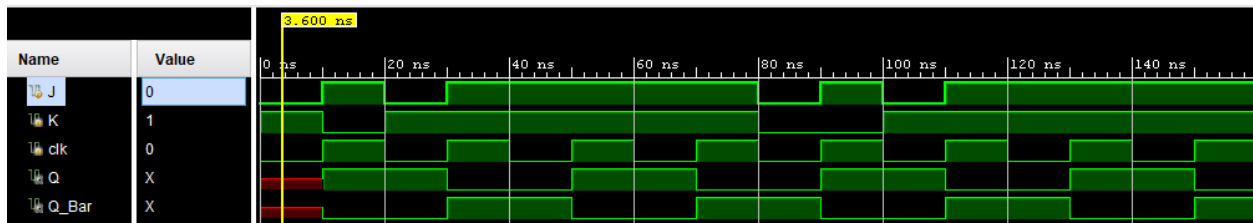
E19 = ~Q



- 2-3 Sentence of test plan

For the JK flip flop simulation, I've tested every iteration for J and K, where clk is set to 0 initially, and flips once every 10 nanoseconds. Each is separated by 10 nanoseconds, and one period of 50 nanoseconds to show that Q toggles when J and K are 1.

- Waveform based on test plan



Part 4:

- Verilog RTL of T flip-flop: The source code for the T flip-flop has 2 inputs (T, Clk) and 2 outputs (Q, Q_Bar). The purpose of the T flip flop is to toggle what Q currently is. I was able to use the JK flip flop to reflect this behavior. Due to the nature of the T flip flop, it's unable to set what Q is, if Q is undefined. I provided an updated JK code where Q starts at 0.

```

1  `timescale 1ns / 1ps
2
3  module t_flip_flop(
4      input T, Clk,
5      output Q, Q_Bar
6  );
7      jk_flip_flop AACT(.J(T), .K(T), .clk(Clk), .Q(Q), .Q_Bar(Q_Bar));
8  endmodule

```

Updated JK flip flop source:

```

1 | `timescale 1ns / 1ps
2 |
3 | module jk_flip_flop(
4 |     input J, K, clk,
5 |     output reg Q,
6 |     output Q_Bar
7 | );
8 |     initial begin
9 |         Q = 1'b0;
10 |     end
11 |
12 |     always @(posedge clk) begin
13 |         case ({J, K})
14 |             2'b00: Q <= Q;
15 |             2'b01: Q <= 0;
16 |             2'b10: Q <= 1;
17 |             2'b11: Q <= ~Q;
18 |         endcase
19 |     end
20 |     assign Q_Bar = ~Q;
21 | endmodule
22 |

```

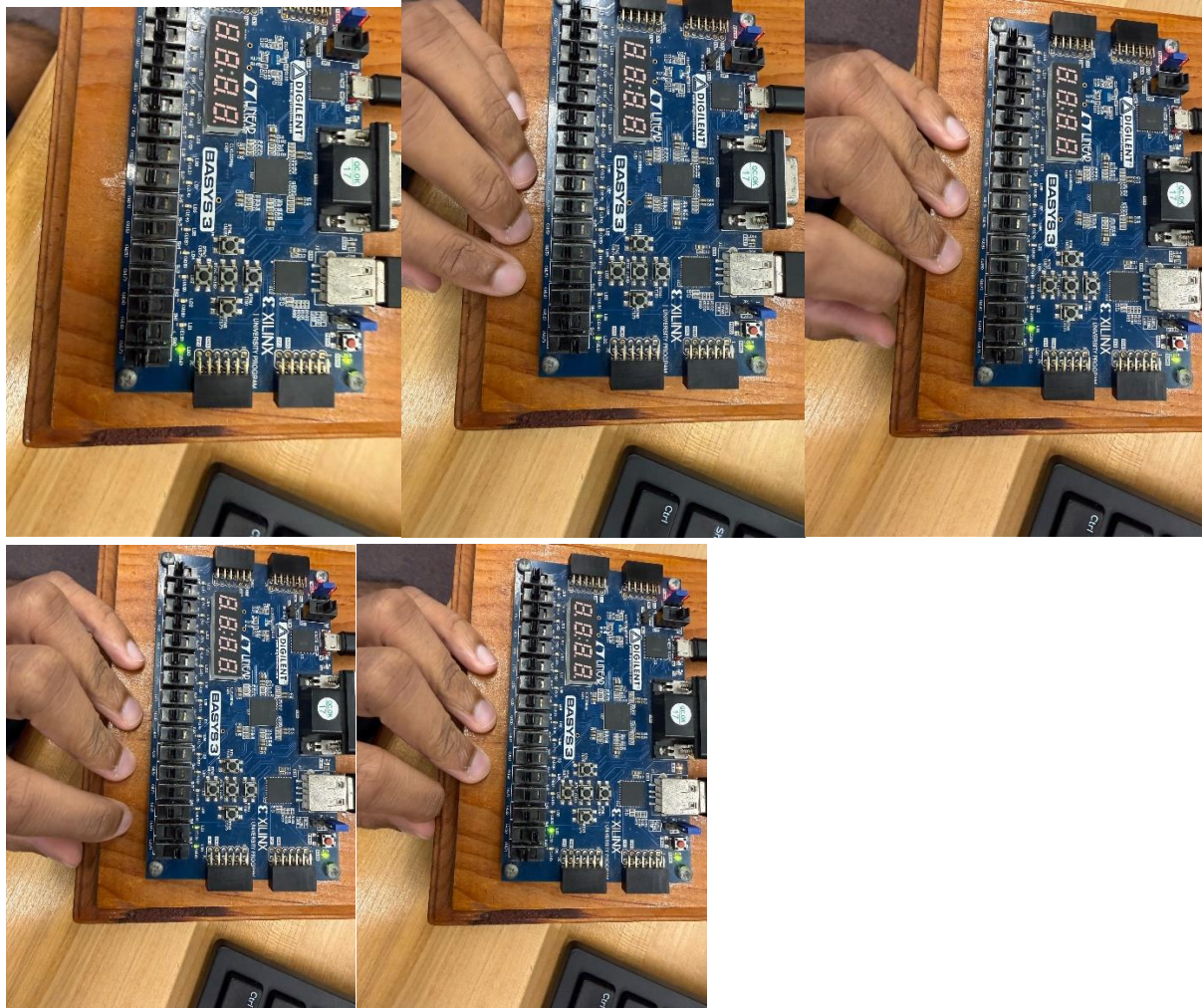
- Pictures of FPGA board for T flip-flop: The fpga board doesn't properly show the expected results. When T is switched on, Q and Q_bar are both shown to turn on. However, since Q can't be set, this would be a limitation of current user knowledge.

W5 = Clk

V17 = T

U16 = Q

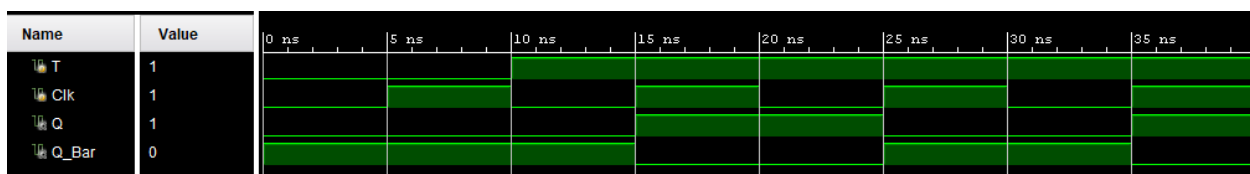
E19 = ~Q



- 2-3 Sentence of test plan

I've set Clk to 0 so T doesn't toggle an undefined number, and I've set Clk to cycle every 5 nanoseconds. T is equal to 0 for 10 nanoseconds, and then T is set to 1 for 30 nanoseconds. To show an output, in the JK Flip flop source code after initializing J, K, clk, reg Q, Q_bar, I added the following code: initial begin Q = 1'b0; end.

- Waveform based on test plan



Part 5:

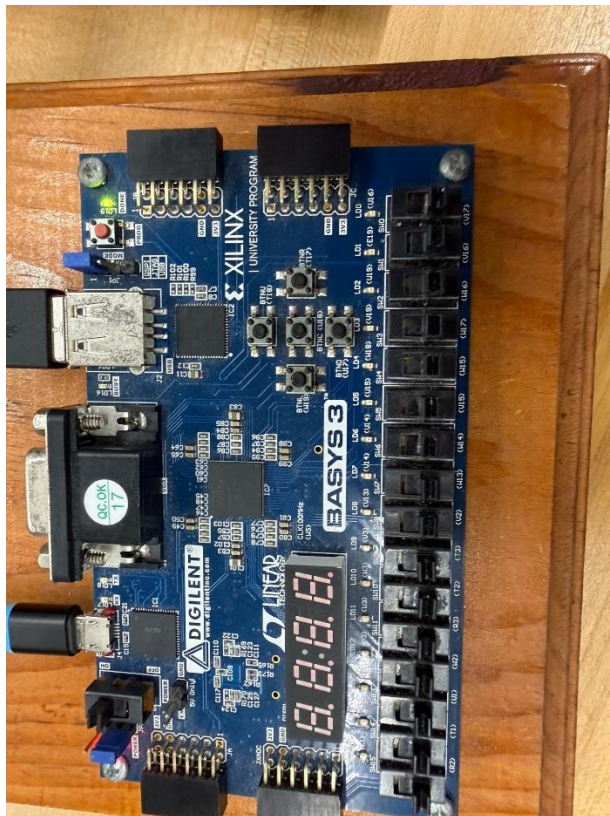
- Verilog RTL of 8-bit dual edge-triggered flip flop: The source code for the 8-bit dual edge-triggered flip flop has 9 inputs([7:0] D, clk) and 16 outputs([7:0] Q, [7:0] Q_Bar). I've used a simple D flip flop logic for this design.

```
1  `timescale 1ns / 1ps
2  //dflip flop method
3  module dual_edge_flip_flop_8bit(
4      input [7:0] D,
5      input clk,
6      output reg [7:0] Q, Q_Bar
7  );
8      always @(clk) begin
9          Q <= D;
10         Q_Bar <= ~D;
11     end
12 endmodule
```

- Pictures of FPGA board for 8-bit dual edge-triggered flip flop

When testing the fpga board, I had to disable the “Q_bar” output due limited number of leds. Since I've used a D flip flop design, flipping a switch would turn on an LED.

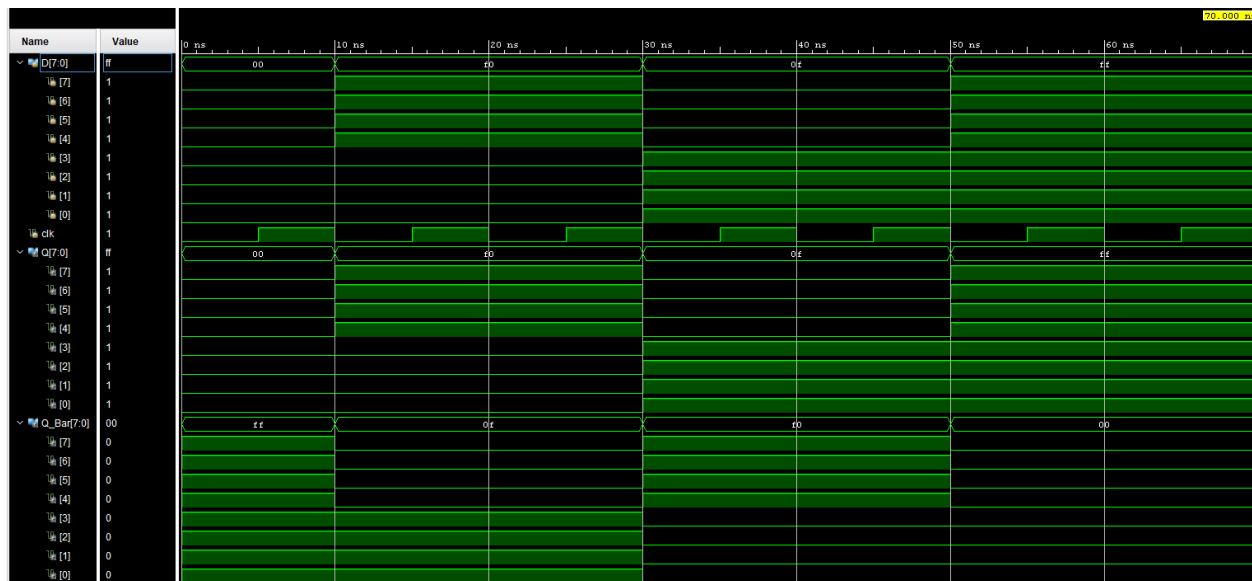
▼ All ports (25)				
▼ D (8)	IN			
D[7]	IN		W13	▼
D[6]	IN		W14	▼
D[5]	IN		V15	▼
D[4]	IN		W15	▼
D[3]	IN		W17	▼
D[2]	IN		W16	▼
D[1]	IN		V16	▼
D[0]	IN		V17	▼
▼ Q (8)	OUT			
Q[7]	OUT		V14	▼
Q[6]	OUT		U14	▼
Q[5]	OUT		U15	▼
Q[4]	OUT		W18	▼
Q[3]	OUT		V19	▼
Q[2]	OUT		U19	▼
Q[1]	OUT		E19	▼
Q[0]	OUT		U16	▼
> Q_Bar (8)	OUT			



- 4-5 Sentence of test plan

In this waveform, clk is set to 0 initially, and toggles every 5 nanoseconds. I tested D = 8'b00000000 for 10 nanoseconds, 8'b11110000 for 20 nanoseconds, 8'b00001111 20 nanoseconds, and 8'b11111111 for 20 nanoseconds. For D = 8'b00000000, Q is set to 8'b00000000, and when D = 8'b11111111, Q is set to 8'b11111111 as well. All in all, the waveform gives the expected result, since Q+ for each bit is the same as D for each bit.

- Waveform based on test plan



Q&A:

What is the difference between a latch and a flip flop?

Latches are asynchronous, where the latch updates when the Enable signal is 1, and flip flops are synchronous, which updates only at the moment of a clock edge occurs, like rising or falling edges.

- What types of flip flops are capable of “toggling”? Explain their differences.

T flip flops and JK flip flops. Toggles occur when T = 1 in any case for T flip flops, and toggles occur when J and K = 1 for JK flip flops.

- How many flip flops would you need if you wanted to make a 32-bit register? What type of flip flop would you use?

You need a flip flop for every register, so 32 flip flops would be needed in this case. D flip flops would be best as it's the simplest.

- Explain in your own words why there is an invalid state within the SR-latch.

There's an invalid state within the SR-latch because the Set makes $Q = 1$, and Reset makes $Q = 0$, and both S and R being 1 creates a contradiction, which makes it invalid.

Test Plan:

Part 1 non gated:

```
1  | `timescale 1ns / 1ps
2  |
3  | module non_gated_SR_latch_sim();
4  | /*module non_gated_SR_latch(
5  |     input S, R,
6  |     output reg Q, Q_bar
7  | );*/
8  |     reg S, R;
9  |     wire Q, Q_bar;
10 |     integer i;
11 |     non_gated_SR_latch UUT(S, R, Q, Q_bar);
12 |     initial begin
13 |         for(i = 0; i<2**2; i = i+1) begin
14 |             {S, R} = i;
15 |             #10;
16 |         end
17 |     end
18 |
19 | endmodule
20 |
```

Gated:

```
1  | `timescale 1ns / 1ps
2  |
3  |
4  | module gated_SR_latch_sim();
5  |     reg S, R, enable;
6  |     wire Q, Q_bar;
7  |     integer i;
8  |     gated_SR_latch AUT(S, R, enable, Q, Q_bar);
9  |     initial begin
10 |         for(i = 0; i<2**3; i = i+1) begin
11 |             {enable, S, R} = i;
12 |             #10;
13 |         end
14 |
15 |     end
16 |
17 | endmodule
```

Part 2: Dflip-flop

```
1  `timescale 1ns / 1ps
2
3
4  module dflip_flop_sim();
5      reg D, clk;
6      wire Q, Q_bar;
7      integer i;
8
9      dflip_flop ABT(D, clk, Q, Q_bar);
10
11  initial begin
12      clk = 0;
13      D = 0;
14      #10;
15      D = 1;
16      #30;
17      clk = 1;
18      D = 0;
19      #40;
20      D = 1;
21      #40;
22      $finish;
23  end
24 endmodule
```

Part 3: JK Flip Flop

```
3 module JK_flip_flop_sim();
4     reg J, K, clk;
5     wire Q, Q_Bar;
6     integer i;
7
8     jk_flip_flop ABT(.J(J), .K(K), .clk(clk), .Q(Q), .Q_Bar(Q_Bar));
9     initial begin
10         clk = 0;
11         J = 0;
12         K = 1;
13         #10;
14         J = 1;
15         K = 0;
16         #10;
17         J = 0;
18         K = 1;
19         #10;
20         J = 1;
21         K = 1;
22         #50;
23
24         J = 0;
25         K = 0;
26         #10;
27         J = 1;
28         K = 0;
29         #10;
30         J = 0;
31         K = 1;
32         #10;
33         J = 1;
34         K = 1;
35         #50;
36         $finish;
37     end
38     always #10 clk = ~clk;
39
40 endmodule
41
```

Part 4:

```
1 `timescale 1ns / 1ps
2
3 module t_flip_flop_slim();
4     reg T, Clk;
5     wire Q, Q_Bar;
6
7     t_flip_flop PARTFOUR(.T(T), .Clk(Clk), .Q(Q), .Q_Bar(Q_Bar));
8     initial begin
9         Clk = 0;
10        T = 0;
11        #10;
12        T = 1;
13        #30;
14        $finish;
15    end
16    always #5 Clk = ~Clk;
17 endmodule
```

Part 5:

```
1      `timescale 1ns / 1ps
2
3      module dual_edge_flip_flop_8bit_sim();
4          reg[7:0] D;
5          reg clk;
6          wire [7:0] Q, Q_Bar;
7
8          dual_edge_flip_flop_8bit PARTFIVE(.D(D), .clk(clk), .Q(Q), .Q_Bar(Q_Bar));
9      initial begin
10         clk = 0;
11         D = 8'b00000000;
12         #10;
13         D = 8'b11110000;
14         #20;
15         D = 8'b00001111;
16         #20;
17         D = 8'b11111111;
18         #20;
19         $finish;
20     end
21     always #5 clk = ~clk;
22
23
24 endmodule
```

Conclusion

This experiment was a success. The results were as expected for all parts of the experiment. I've had a few issues with how the source code is supposed to look, due to lack of knowledge of the syntax I can use. But once I figured it out, it wasn't too difficult. Playing around with the different latches and flip flops to this extent using Vivado and FPGA board has furthered my understanding of the logic, and how useful it can be.